

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler

Muhammad Abdullah - 5024241092

2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler merupakan komponen utama dalam sistem tertanam (*embedded system*) yang memungkinkan suatu perangkat menerima masukan, mengolah data, dan menghasilkan keluaran secara otomatis berdasarkan program yang dijalankan. Penerapannya sangat luas, mulai dari sistem kendali otomatis, perangkat Internet of Things (IoT), sistem monitoring, hingga perangkat rumah pintar. Oleh karena itu, pemahaman mengenai arsitektur mikrokontroler, konfigurasi GPIO, dan lingkungan pengembangannya menjadi dasar yang penting sebelum merancang sistem yang lebih kompleks.

Pada praktikum Modul 1 digunakan dua board, yaitu STM32 Blackpill dan ESP32 DevKit. Keduanya memiliki karakteristik perangkat keras dan framework yang berbeda. STM32 Blackpill diprogram menggunakan framework Arduino melalui programmer ST-Link, sedangkan ESP32 diprogram menggunakan framework Arduino maupun ESP-IDF melalui kabel USB. Visual Studio Code dengan ekstensi PlatformIO digunakan sebagai lingkungan pengembangan untuk menulis, mengompilasi, dan mengunggah program ke kedua board tersebut.

Praktikum ini mencakup pengendalian LED melalui GPIO, pembacaan tombol dengan konfigurasi *pull-up* dan *pull-down* eksternal maupun internal, penanganan *bouncing* dengan metode *debouncing*, serta pembacaan sinyal analog menggunakan ADC dan sensor LDR. Laporan sementara ini memuat latar belakang, dasar teori, dan jawaban tugas pendahuluan sebagai bekal sebelum kegiatan praktikum dilaksanakan.

1.2 Dasar Teori

1. **Mikrokontroler dan GPIO**
   Mikrokontroler adalah sistem komputer berukuran kecil yang mengintegrasikan prosesor, memori, dan peripheral input/output dalam satu chip. GPIO (*General Purpose Input/Output*) merupakan pin yang dapat dikonfigurasi sebagai digital output, digital input, atau analog input. Digital output digunakan untuk menghasilkan level HIGH/LOW, digital input untuk membaca kondisi logika, sedangkan analog input memanfaatkan ADC untuk membaca tegangan yang berubah secara kontinu.

2. **PlatformIO dan Visual Studio Code**
   PlatformIO adalah ekosistem pengembangan *embedded* yang mendukung banyak board dan framework dalam satu lingkungan kerja. Integrasinya dengan Visual Studio Code memudahkan proses penulisan kode, *build*, dan *upload*. Struktur project PlatformIO terdiri atas `platformio.ini` sebagai konfigurasi board dan framework, folder `src` untuk program utama, folder `include` untuk header, dan folder `lib` untuk library tambahan.

3. **STM32 Blackpill dan Framework Arduino**
   STM32 Blackpill berbasis mikrokontroler STM32F401CCU6 dengan arsitektur ARM Cortex-M4 32-bit, clock hingga 84 MHz, Flash 256 KB, dan RAM 64 KB. LED onboard terhubung ke pin PC13 dengan konfigurasi aktif LOW. Board ini tidak memiliki programmer onboard sehingga memerlukan ST-Link melalui protokol SWD. Framework Arduino (STM32duino) menyediakan fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()` sehingga pemrograman menjadi lebih sederhana.

4. **ESP32 dan Framework ESP-IDF**
   ESP32 merupakan mikrokontroler 32-bit *dual-core* Xtensa LX6 dengan WiFi dan Bluetooth terintegrasi serta USB-to-Serial bawaan. ESP-IDF adalah framework resmi Espressif yang dibangun di atas FreeRTOS dan menggunakan `app_main()` sebagai titik masuk program. Berbeda dengan framework Arduino yang menggunakan `setup()` dan `loop()`, ESP-IDF memberikan akses langsung terhadap task, semaphore, dan queue FreeRTOS.

5. **Pull-up dan Pull-down Resistor**
   Pin GPIO yang tidak terhubung ke level tegangan tertentu berada pada kondisi *floating* sehingga nilainya dapat berubah acak akibat noise. Pull-up resistor menghubungkan pin ke VCC melalui resistor, sehingga pin terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan. Pull-down resistor menghubungkan pin ke GND, sehingga pin terbaca LOW saat tombol tidak ditekan dan HIGH saat ditekan. Kedua konfigurasi dapat dipasang secara eksternal maupun diaktifkan secara internal melalui `INPUT_PULLUP` dan `INPUT_PULLDOWN`.

6. **Debouncing**
   Tombol mekanik menghasilkan getaran kontak saat ditekan atau dilepaskan sehingga sinyal dapat berubah HIGH–LOW beberapa kali dalam waktu singkat. Fenomena ini disebut *bouncing* dan dapat menyebabkan satu penekanan terbaca sebagai beberapa event. *Debouncing* memastikan perubahan status hanya diterima setelah sinyal stabil selama periode tertentu, umumnya 20–50 ms.

7. **ADC dan LDR**
   ADC (*Analog-to-Digital Converter*) mengubah tegangan analog menjadi nilai digital dengan cara melakukan pencuplikan, kuantisasi, dan pengodean terhadap tegangan masukan. ESP32 memiliki ADC 12-bit dengan rentang nilai 0–4095 dan referensi sekitar 3,3 V. Light Dependent Resistor (LDR) memiliki resistansi yang menurun ketika intensitas cahaya meningkat. Pada praktikum ini LDR dirangkai sebagai pembagi tegangan bersama resistor 10 kΩ, dan titik tengahnya dihubungkan ke GPIO 34 untuk dibaca melalui `analogRead()`.

2 Tugas Pendahuluan

1. **Perbedaan framework Arduino dan ESP-IDF**

   Framework Arduino memiliki tingkat abstraksi tinggi dan menggunakan struktur `setup()`–`loop()` sehingga mudah dipelajari, terutama bagi pemula. Fungsi seperti `pinMode()`, `digitalWrite()`, dan `delay()` menyederhanakan akses perangkat keras. Kekurangannya, kontrol terhadap peripheral dan RTOS lebih terbatas serta kurang efisien untuk sistem yang kompleks.

   ESP-IDF merupakan framework native Espressif berbasis FreeRTOS dengan `app_main()` sebagai titik masuk program. Framework ini memberi akses langsung terhadap task, semaphore, queue, dan konfigurasi peripheral tingkat rendah sehingga lebih fleksibel dan sesuai untuk pengembangan IoT tingkat lanjut. Kekurangannya, struktur program lebih kompleks dan membutuhkan pemahaman RTOS serta arsitektur ESP32 yang lebih mendalam.

2. **Alasan STM32 Blackpill memerlukan ST-Link**

STM32F401CCU6 tidak memiliki chip USB-to-Serial maupun bootloader USB bawaan pada board Blackpill. Akibatnya, program tidak dapat diunggah langsung melalui kabel USB seperti pada board Arduino. ST-Link diperlukan sebagai programmer/debugger eksternal yang berkomunikasi melalui protokol SWD (SWDIO, SWCLK, GND, dan 3,3 V) untuk menulis program ke Flash STM32.

Sebaliknya, ESP32 DevKit telah dilengkapi chip USB-to-Serial (misalnya CP2102 atau CH340) dan bootloader ROM di dalam chip. Chip USB-to-Serial mengubah data dari USB menjadi sinyal UART yang dipahami ESP32, sehingga proses *flashing* dapat dilakukan langsung melalui kabel USB tanpa programmer tambahan.

3. **Kondisi logika pull-up dan pull-down**

Tabel 1: Kondisi logika pin GPIO pada konfigurasi pull-up dan pull-down

Konfigurasi	Tombol Tidak Ditekan	Tombol Ditekan
Pull-up	HIGH	LOW
Pull-down	LOW	HIGH

Pada pull-up, pin ditarik ke VCC sehingga bernilai HIGH saat tombol tidak ditekan; ketika tombol ditekan, pin terhubung ke GND dan bernilai LOW. Pada pull-down berlaku kebalikannya, yaitu pin ditarik ke GND sehingga bernilai LOW saat tidak ditekan dan menjadi HIGH saat tombol menghubungkan pin ke VCC. Konfigurasi ini mencegah kondisi *floating* yang membuat pembacaan tombol tidak stabil.

4. **Fenomena bouncing dan cara mengatasinya**

Bouncing terjadi karena kontak mekanik pada tombol tidak langsung menempel sempurna saat ditekan atau dilepaskan. Getaran fisik tersebut menghasilkan beberapa transisi HIGH–LOW dalam waktu sangat singkat, sehingga satu penekanan dapat terbaca sebagai beberapa kali penekanan. Jika tidak ditangani, counter atau aksi yang dipicu tombol dapat bertambah lebih dari satu.

Beberapa pendekatan untuk mengatasinya adalah:

(a) *Debouncing* perangkat lunak, yaitu menunggu sinyal stabil selama periode tertentu (misalnya 20–50 ms) sebelum perubahan status diterima, dapat menggunakan `delay()` sederhana maupun state machine berbasis `millis()`.

(b) *Debouncing* perangkat keras, yaitu menambahkan rangkaian RC *low-pass filter* dan/atau Schmitt trigger agar transisi sinyal menjadi lebih landai dan bersih.

(c) Penggunaan library debouncing atau pembacaan berkala dengan timer/interrupt yang memvalidasi kestabilan sinyal sebelum dianggap sebagai satu event.

5. **Cara kerja ADC dan pengaruh resolusi bit**

ADC mengubah sinyal analog menjadi digital melalui tiga tahap: *sampling* (mencuplik tegangan pada waktu tertentu), *quantization* (membulatkan nilai tegangan ke level diskret terdekat), dan *encoding* (mengubah level tersebut menjadi kode biner). Nilai digital hasil konversi dibandingkan dengan tegangan referensi (V_{ref}) dan dirumuskan sebagai:

$$\text{Nilai ADC} = \frac{V_{in}}{V_{ref}} \times (2^N - 1) \tag{1}$$

dengan N adalah resolusi ADC dalam bit.

Resolusi menentukan jumlah level diskret yang dapat direpresentasikan, yaitu 2^N. ADC 12-bit pada ESP32 memiliki 4096 level (0–4095). Dengan $V_{ref} = 3{,}3$ V, perubahan tegangan terkecil yang dapat dibedakan kira-kira $3{,}3/4096 \approx 0{,}81$ mV. Semakin tinggi resolusi ADC, semakin kecil langkah kuantisasi sehingga hasil pembacaan semakin presisi, tetapi jumlah data dan kompleksitas pengolahan juga meningkat.

6. **Setup lingkungan pengembangan**

   Lingkungan pengembangan telah disiapkan pada laptop dengan langkah berikut:

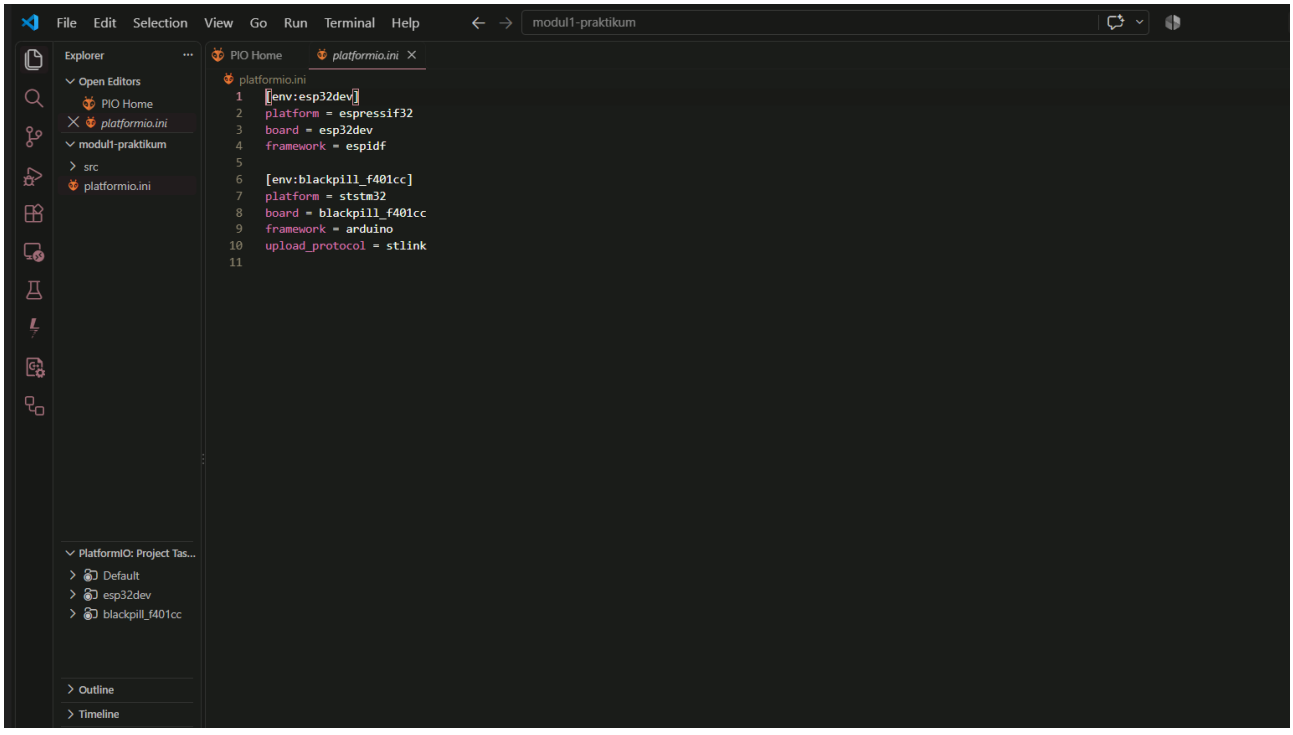
   (a) Memasang Visual Studio Code beserta ekstensi PlatformIO IDE.

   (b) Memasang platform `espressif32` untuk board ESP32 dengan framework ESP-IDF dan platform `ststm32` untuk board STM32 Blackpill dengan framework Arduino.

   (c) Memasang ekstensi Serial Plotter untuk memantau data serial dalam bentuk grafik.

   (d) Membuat project `modul1-praktikum` dengan dua environment, yaitu `esp32dev` (ESP-IDF) dan `blackpill_f401cc` (Arduino), untuk memastikan kedua framework dapat dikenali oleh PlatformIO.

   Hasil setup ditunjukkan pada Gambar 1. Project berhasil dikenali oleh PlatformIO dan menampilkan *Project Tasks* untuk kedua environment.



Gambar 1: Tampilan Visual Studio Code dengan project PlatformIO yang memuat environment ESP32 (ESP-IDF) dan STM32 Blackpill (Arduino).